



ASCI

Encapsulation



Pengertian

Encapsulation adalah konsep OOP yang membungkus atribut (data) dan method dalam sebuah *class*, hal ini juga sekaligus menyembunyikan detail implementasi internal dari akses luar yang tidak seharusnya. Tujuan utamanya adalah melindungi data agar tidak diubah secara sembarangan dari luar *class* dan hanya dapat diakses melalui method yang telah disediakan.

Dengan adanya *Encapsulation*, sebuah *class* jadi dapat mengatur:

1. Apa yang bisa diakses dari luar (*public*)
2. Apa yang hanya bisa diakses dari dalam *class/subclass* (*Protected*)
3. Apa yang benar-benar tersembunyi, hanya bisa diakses dari dalam *class* itu sendiri (*Private*)

Penerapan Encapsulation

Encapsulation diterapkan agar suatu *class* bisa mengontrol validitas datanya sendiri. Misalnya, atribut **SALDO** pada *class RekeningBank* tidak boleh diubah secara langsung menjadi sebuah nilai negatif dari luar *class*. Tanpa *Encapsulation*, siapa pun bisa langsung menulis “*rekening.saldo = -500000*”, yang sangat jelas tidak boleh dan tidak valid secara bisnis.

```
class RekeningBankTanpaEncapsulation:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.saldo = saldo

rekening = RekeningBankTanpaEncapsulation("Budi", 100000)
rekening.saldo = -500000    # tidak ada yang mencegah ini
print(rekening.saldo)      # -500000, data jadi tidak valid
```



Access Modifier di Python

Berbeda dengan **Java** yang menggunakan *keyword* seperti **Public**, **Private**, dan **Protected**, **Python** menerapkan Encapsulation melalui penamaan konvensi menggunakan garis bawah (*underscore*) (`_`), hal ini berkaitan dengan filosofi **Python** yang menganut prinsip “*we’re all consenting adults here*”. Tidak ada pembatasan akses yang benar-benar dipaksakan oleh bahasa, tapi konvensinya tetap dihormati.

Modifier	Penulisan	Dapat diakses dari
<i>Public</i>	nama_atribut	Dimana saja (dalam & luar <i>class</i>)
<i>Protected</i>	<code>_nama_atribut</code>	Dalam <i>class</i> & <i>subclass</i> secara konvensi
<i>Private</i>	<code>__nama_atribut</code>	Hanya didalam <i>class</i> itu sendiri (<i>name mangling</i>)

Public

Atribut/Method *Public* bisa diakses secara bebas dari luar *class*, tanpa aturan penamaan khusus.

```
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama    # public
        self.nim = nim      # public

mhs = Mahasiswa("Dap", "2409106050")
print(mhs.nama)    # bisa diakses langsung
print(mhs.nim)
```



Protected

Atribut *Protected* ditandai dengan satu garis bawah pada bagian depan nama (**`_nama_atribut`**). Secara teknis masih dapat diakses dari luar class, tapi ini merupakan sinyal untuk developer lain bahwa atribut ini hanya dimaksudkan untuk dipakai di dalam *class* itu sendiri atau class turunannya (*subclass*).

```
class Karyawan:
    def __init__(self, nama, gaji):
        self.nama = nama
        self._gaji = gaji  # protected, hanya "disarankan" diakses dari
dalam

class Manager(Karyawan):
    def tampilkan_gaji(self):
        # subclass tetap bisa mengakses atribut protected milik parent
        print(f"Gaji {self.nama}: {self._gaji}")

manager = Manager("Daffa", 12000000)
manager.tampilkan_gaji()
print(manager._gaji)  # masih bisa diakses, tapi secara konvensi sebaiknya
tidak
```

Private

Atribut *Private* ditandai dengan Dua garis bawah pada bagian depan nama (**`__nama_atribut`**). Pada bagian ini, **Python** akan melakukan **name mangling**, yaitu mengubah nama atribut secara internal menjadi (**`_NamaClass__nama_atribut`**), hal ini menyebabkan tidak bisa diakses secara langsung dari luar *class* dengan nama aslinya.



```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.__saldo = saldo # private

    def tarik_saldo(self, jumlah):
        if jumlah > self.__saldo:
            print("Saldo tidak cukup.")
        elif jumlah <= 0:
            print("Jumlah penarikan tidak valid.")
        else:
            self.__saldo -= jumlah
            print(f"Berhasil menarik {jumlah}. Sisa saldo: {self.__saldo}")

    def cek_saldo(self):
        print(f"Saldo saat ini: {self.__saldo}")

rekening = RekeningBank("Budi", 100000)
rekening.tarik_saldo(30000)
rekening.cek_saldo()

print(rekening.__saldo) # AttributeError, karena sudah di-name-mangling
```

```
# akses "paksa" ke private tetap mungkin lewat name mangling, tapi ini
# melanggar konvensi encapsulation dan sebaiknya TIDAK dilakukan:
print(rekening._RekeningBank__saldo) # 70000, tapi ini praktik yang buruk
```

Getter & Setter

Karena atribut *Private* dan *Protected* tidak boleh diakses/diubah secara langsung dari luar, maka *encapsulation* biasanya dilengkapi dengan method *Getter* (untuk mengambil dan membaca nilai) dan *Setter* (untuk melakukan perubahan nilai dengan validasi). **Python** memiliki cara lebih idiomatis dibanding **Java** untuk hal ini, yaitu menggunakan decorator “**@Property**”.



Cara manual menggunakan *Getter* dan *Setter* seperti java

```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.__saldo = saldo

    def get_saldo(self):
        return self.__saldo

    def set_saldo(self, saldo_baru):
        if saldo_baru < 0:
            print("Saldo tidak boleh negatif.")
        else:
            self.__saldo = saldo_baru

rekening = RekeningBank("Budi", 100000)
print(rekening.get_saldo())
rekening.set_saldo(-500)      # ditolak oleh validasi
rekening.set_saldo(200000)   # diterima
print(rekening.get_saldo())
```

Cara idiomatis **Python** menggunakan decorator

```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.__saldo = saldo

    @property
    def saldo(self):
        """Getter -- dipanggil seperti atribut biasa, tanpa tanda
        kurung."""
        return self.__saldo

    @saldo.setter
    def saldo(self, saldo_baru):
```



```
        """Setter -- dijalankan otomatis saat ada assignment ke
rekening.saldo"""
        if saldo_baru < 0:
            raise ValueError("Saldo tidak boleh negatif.")
        self.__saldo = saldo_baru

rekening = RekeningBank("Budi", 100000)
print(rekening.saldo)           # dipanggil seperti atribut, bukan method
rekening.saldo = 250000         # otomatis lewat setter dengan validasi
print(rekening.saldo)

rekening.saldo = -1000          # akan raise ValueError
```